

Reinforcement Learning Laboratory

Lab Assignment 3: GridWorld – Policy Evaluation and Value Iteration

Objective

Formulate a GridWorld environment as a Reinforcement Learning problem, implement Policy Evaluation and Value Iteration, and compare different policies (random, evaluated, and optimal).

Environment Used

A 2×3 GridWorld with 5 non-terminal states (S1–S5) and one terminal Goal state, laid out as:

S1 S2 S3

S4 S5 GOAL

Discount factor $\gamma = 0.9$. Every move (including bumping into a wall, which leaves the agent in place) gives a reward of -1 ; reaching GOAL gives $+10$ and ends the episode.

Task 1: Formulating the GridWorld Environment

RL Component	Description
States	5 non-terminal grid cells (S1–S5) plus 1 terminal Goal state, arranged in a 2×3 grid.
Actions	UP, DOWN, LEFT, RIGHT — attempting to move off the grid boundary leaves the agent in the same cell.
Reward Structure	-1 for every step taken (including wall bumps); $+10$ for reaching the Goal state.
Terminal State(s)	GOAL (bottom-right cell). The episode ends immediately once it is reached.
Goal of the Agent	Learn a policy that reaches GOAL from any starting state while maximizing cumulative discounted reward (equivalently, in the fewest steps).

Task 2: Policy Evaluation

Policy evaluated: a fixed heuristic policy π that moves RIGHT whenever not in the rightmost column, and DOWN otherwise.

Iteration	Maximum Change (Δ)	Convergence Status
1	10.000000	Not yet converged
2	9.000000	Not yet converged
3	8.100000	Not yet converged
4	0.000000	Converged ($\Delta < 1e-4$)

Resulting state values under the evaluated policy ($\gamma = 0.9$):

State	Action Taken	State Value $V(s)$
S1	RIGHT	6.200
S2	RIGHT	8.000
S3	DOWN	10.000
S4	RIGHT	8.000
S5	RIGHT	10.000

Task 3: Value Iteration and Optimal Policy Generation

Value iteration converged in 4 iterations ($\theta = 1e-4$).

State	Optimal Action	State Value
S1	DOWN	6.200
S2	DOWN	8.000
S3	DOWN	10.000
S4	RIGHT	8.000
S5	RIGHT	10.000

Note: the optimal state values are identical to those found under the evaluated policy in Task 2 — the heuristic policy (RIGHT-then-DOWN) already happens to be optimal for this grid, though S1 admits two equally optimal actions (RIGHT and DOWN, both reaching GOAL in 3 steps).

Task 4: Optimal Path Analysis

Policy Type	Path Length	Total Reward	Goal Reached (Yes/No)
Random Policy	17	-6	Yes
Evaluated Policy	3	8	Yes
Optimal Policy	3	8	Yes

Sample trajectories:

- Random policy: S1 → S1 → S1 → S1 → S4 → S4 → S4 → S1 → S1 → S2 → S2 → S2 → S2 → S5 → S5 → S2 → S5 → GOAL (17 steps, reward = -6)
- Evaluated policy: S1 → S2 → S3 → GOAL (3 steps, reward = 8)
- Optimal policy: S1 → S4 → S5 → GOAL (3 steps, reward = 8)

Analysis Questions

1. What is a state in GridWorld?

A state represents the agent's current position on the grid — one specific cell (S1–S5, or the terminal GOAL cell). It fully describes the situation the agent is in at a given time step.

2. What actions can the agent perform?

The agent can move UP, DOWN, LEFT, or RIGHT. If an action would take it outside the grid boundary, it simply stays in its current cell (a wall bump) and still incurs the step cost.

3. What is the purpose of Policy Evaluation?

Policy Evaluation computes the state-value function $V(s)$ for a fixed, given policy — i.e., it answers 'how good is each state if the agent keeps following this particular policy?' It does this by repeatedly applying the Bellman expectation equation until the values stop changing (converge), without changing the policy itself.

4. How does Value Iteration differ from Policy Evaluation?

Policy Evaluation computes values for one fixed policy and never changes the policy. Value Iteration instead updates each state's value using the BEST available action at every sweep (a max over actions rather than an average/fixed choice), so it simultaneously improves the policy and evaluates it. In effect,

Value Iteration combines policy evaluation and policy improvement into a single step and converges directly to the optimal value function V^* .

5. What is the Bellman Optimality Equation?

It states that the optimal value of a state equals the expected return of the best action taken from that state: $V^*(s) = \max_a [R(s,a) + \gamma \cdot V^*(s')]$, where s' is the resulting next state. It expresses that an optimal policy always chooses the action that maximizes immediate reward plus the discounted value of the state it leads to.

6. Which policy reached the goal using the fewest steps?

Both the Evaluated Policy and the Optimal Policy reached the goal in the fewest steps (3 steps each). The Random Policy took considerably longer (17 steps in the sampled run) because it has no directional preference and can wander or bump into walls repeatedly.

7. Why is the optimal policy superior to a random policy?

The optimal policy always selects the action with the highest expected discounted return at every state, so it reaches the goal via the shortest possible path and maximizes cumulative reward. The random policy has no knowledge of the environment's structure, so its path length and reward are highly variable and, on average, much worse — as seen in Task 4 (reward of -6 vs. $+8$).

8. How would obstacles affect the optimal path?

Obstacles would remove or block certain state transitions (an action leading into an obstacle would behave like a wall bump, or the obstacle cell could be excluded from the state space entirely). Value Iteration would naturally route around them, since paths through obstacles would never yield the maximum value — the optimal path could become longer than the straight-line Manhattan distance, and in a maze-like configuration multiple detours might tie for optimality.

Conclusion

Policy Evaluation gave a systematic way to quantify how good a fixed policy is at every state, by repeatedly applying the Bellman expectation equation until the value function converged (4 iterations here). Value Iteration went a step further: by taking the max over actions at each sweep instead of following a fixed policy, it directly produced the optimal value function and, by acting greedily with respect to it, the optimal policy — also in 4 iterations for this small grid. Comparing policies confirmed the expected ordering: the random policy reached the goal but inefficiently (17 steps, reward -6), while both the evaluated heuristic policy and the value-iteration-derived optimal policy reached the goal along the shortest possible path (3 steps, reward $+8$). In this particular GridWorld the hand-designed heuristic happened to already be optimal, but Value Iteration is what guarantees optimality in general, especially once the environment becomes more complex (e.g., with obstacles, stochastic transitions, or a larger state space) where a hand-crafted policy could no longer be trusted to be best.